2.13 stdlib.h

The stdlib header defines several general operation functions and macros.

Macros:

```
NULL
EXIT_FAILURE
EXIT_SUCCESS
RAND_MAX
MB_CUR_MAX
```

Variables:

```
typedef size_t
typedef wchar_t
struct div_t
struct ldiv_t
```

Functions:

```
abort();
abs();
atexit();
atof();
atoi();
atol();
bsearch();
calloc();
div();
exit();
free();
getenv();
labs();
ldiv();
malloc();
mblen();
mbstowcs();
mbtowc();
qsort();
rand();
realloc();
srand();
strtod();
strtol();
strtoul();
system();
wcstombs();
wctomb();
```

2.13.1 Variables and Definitions

`size_t` is the unsigned integer result of the `sizeof` keyword.
`wchar_t` is an integer type of the size of a wide character constant.

`div_t` is the structure returned by the `div` function.
`ldiv_t` is the structure returned by the `ldiv` function.

`NULL` is the value of a null pointer constant.
`EXIT_FAILURE` and `EXIT_SUCCESS` are values for the exit function to return termination status.
`RAND_MAX` is the maximum value returned by the rand function.
`MB_CUR_MAX` is the maximum number of bytes in a multibyte character set which cannot be larger than `MB_LEN_MAX`.

2.13.2 String Functions

2.13.2.1 atof

Declaration:

```
double atof(const char *str);
```

The string pointed to by the argument *str* is converted to a floating-point number (type `double`). Any initial whitespace characters are skipped (space, tab, carriage return, new line, vertical tab, or formfeed). The number may consist of an optional sign, a string of digits with an optional decimal character, and an optional `e` or `E` followed by a optionally signed exponent. Conversion stops when the first unrecognized character is reached.

On success the converted number is returned. If no conversion can be made, zero is returned. If the value is out of range of the type double, then `HUGE_VAL` is returned with the appropriate sign and `ERANGE` is stored in the variable `errno`. If the value is too small to be returned in the type `double`, then zero is returned and `ERANGE` is stored in the variable `errno`.

2.13.2.2 atoi

Declaration:

```
int atoi(const char *str);
```

The string pointed to by the argument *str* is converted to an integer (type `int`). Any initial whitespace characters are skipped (space, tab, carriage return, new line, vertical tab, or formfeed). The number may consist of an optional sign and a string of digits. Conversion stops when the first unrecognized character is reached.

On success the converted number is returned. If the number cannot be converted, then `0` is returned.

2.13.2.3 atol

Declaration:

```
long int atol(const char *str);
```

The string pointed to by the argument *str* is converted to a long integer (type `long int`). Any initial whitespace characters are skipped (space, tab, carriage return, new line, vertical tab, or formfeed). The number may consist of an optional sign and a string of digits. Conversion stops when the first unrecognized character is reached.

On success the converted number is returned. If the number cannot be converted, then `0` is returned.

2.13.2.4 strtod

Declaration:

```
double strtod(const char *str, char **endptr);
```

The string pointed to by the argument *str* is converted to a floating-point number (type `double`). Any initial whitespace characters are skipped (space, tab, carriage return, new line, vertical tab, or formfeed). The number may consist of an optional sign, a string of digits with an optional decimal character, and an optional `e` or `E` followed by a optionally signed exponent. Conversion stops when the first unrecognized character is reached.

The argument *endptr* is a pointer to a pointer. The address of the character that stopped the scan is stored in the pointer that *endptr* points to.

On success the converted number is returned. If no conversion can be made, zero is returned. If the value is out of range of the type double, then `HUGE_VAL` is returned with the appropriate sign and `ERANGE` is stored in the variable `errno`. If the value is too small to be returned in the type `double`, then zero is returned and `ERANGE` is stored in the variable `errno`.

2.13.2.5 strtol

Declaration:

```
long int strtol(const char *str, char **endptr, int base);
```

The string pointed to by the argument *str* is converted to a long integer (type `long int`). Any initial whitespace characters are skipped (space, tab, carriage return, new line, vertical tab, or formfeed). The number may consist of an optional sign and a string of digits. Conversion stops when the first unrecognized character is reached.

If the *base* (radix) argument is zero, then the conversion is dependent on the first two characters. If the first character is a digit from 1 to 9, then it is base 10. If the first digit is a zero and the second digit is a digit from 1 to 7, then it is base 8 (octal). If the first digit is a zero and the second character is an x or X, then it is base 16 (hexadecimal).

If the *base* argument is from 2 to 36, then that base (radix) is used and any characters that fall outside of that base definition are considered unconvertible. For base 11 to 36, the characters A to Z (or a to z) are used. If the base is 16, then the characters 0x or 0X may precede the number.

The argument *endptr* is a pointer to a pointer. The address of the character that stopped the scan is stored in the pointer that *endptr* points to.

On success the converted number is returned. If no conversion can be made, zero is returned. If the value is out of the range of the type `long int`, then `LONG_MAX` or `LONG_MIN` is returned with the sign of the correct value and `ERANGE` is stored in the variable `errno`.

2.13.2.6 strtoul

Declaration:

```
unsigned long int strtoul(const char *str, char **endptr, int base);
```

The string pointed to by the argument *str* is converted to an unsigned long integer (type `unsigned long int`). Any initial whitespace characters are skipped (space, tab, carriage return, new line, vertical tab, or formfeed). The number may consist of an optional sign and a string of digits. Conversion stops when the first unrecognized character is reached.

If the *base* (radix) argument is zero, then the conversion is dependent on the first two characters. If the first character is a digit from 1 to 9, then it is base 10. If the first digit is a zero and the second digit is a digit from 1 to 7, then it is base 8 (octal). If the first digit is a zero and the second character is an x or X, then it is base 16 (hexadecimal).

If the *base* argument is from 2 to 36, then that base (radix) is used and any characters that fall outside of that base definition are considered unconvertible. For base 11 to 36, the characters A to Z (or a to z) are used. If the *base* is 16, then the characters 0x or 0X may precede the number.

The argument *endptr* is a pointer to a pointer. The address of the character that stopped the scan is stored in the pointer that *endptr* points to.

On success the converted number is returned. If no conversion can be made, zero is returned. If the value is out of the range of the type `unsigned long int`, then `ULONG_MAX` is returned and `ERANGE` is stored in the variable `errno`.

2.13.3 Memory Functions

2.13.3.1 calloc

Declaration:

```
void *calloc(size_t nitems, size_t size);
```

Allocates the requested memory and returns a pointer to it. The requested size is *nitems* each *size* bytes long (total memory requested is nitems*size). The space is initialized to all zero bits.

On success a pointer to the requested space is returned. On failure a null pointer is returned.

2.13.3.2 free

Declaration:

```
void free(void *ptr);
```

Deallocates the memory previously allocated by a call to `calloc`, `malloc`, or `realloc`. The argument *ptr* points to the space that was previously allocated. If *ptr* points to a memory block that was not allocated with `calloc`, `malloc`, or `realloc`, or is a space that has been deallocated, then the result is undefined.

No value is returned.

2.13.3.3 malloc

Declaration:

```
void *malloc(size_t size);
```

Allocates the requested memory and returns a pointer to it. The requested size is *size* bytes. The value of the space is indeterminate.

On success a pointer to the requested space is returned. On failure a null pointer is returned.

2.13.3.4 realloc

Declaration:

```
void *realloc(void *ptr, size_t size);
```

Attempts to resize the memory block pointed to by *ptr* that was previously allocated with a call to `malloc` or `calloc`. The contents pointed to by *ptr* are unchanged. If the value of *size* is greater than the previous size of the block, then the additional bytes have an undeterminate value. If the value of *size* is less than the previous size of the block, then the difference of bytes at the end of the block are freed. If ptr is null, then it behaves like `malloc`. If *ptr* points to a memory block that was not allocated with `calloc` or `malloc`, or is a space that has been deallocated, then the result is undefined. If the new space cannot be allocated, then the contents pointed to by *ptr* are unchanged. If size is zero, then the memory block is completely freed.

On success a pointer to the memory block is returned (which may be in a different location as before). On failure or if size is zero, a null pointer is returned.

2.13.4 Environment Functions

2.13.4.1 abort

Declaration:

```
void abort(void);
```

Causes an abnormal program termination. Raises the `SIGABRT` signal and an unsuccessful termination status is returned to the environment. Whether or not open streams are closed is implementation-defined.

No return is possible.

2.13.4.2 atexit

Declaration:

```
int atexit(void (*func)(void));
```

Causes the specified function to be called when the program terminates normally. At least 32 functions can be registered to be called when the program terminates. They are called in a last-in, first-out basis (the last function registered is called first).

On success zero is returned. On failure a nonzero value is returned.

2.13.4.3 exit

Declaration:

```
void exit(int status);
```

Causes the program to terminate normally. First the functions registered by atexit are called, then all open streams are flushed and closed, and all temporary files opened with tmpfile are removed. The value of *status* is returned to the environment. If *status* is **EXIT_SUCCESS**, then this signifies a successful termination. If *status* is **EXIT_FAILURE**, then this signifies an unsuccessful termination. All other values are implementation-defined.

No return is possible.

2.13.4.4 getenv

Declaration:

```
char *getenv(const char *name);
```

Searches for the environment string pointed to by *name* and returns the associated value to the string. This returned value should not be written to.

If the string is found, then a pointer to the string's associated value is returned. If the string is not found, then a null pointer is returned.

2.13.4.5 system

Declaration:

```
int system(const char *string);
```

The command specified by string is passed to the host environment to be executed by the command

processor. A null pointer can be used to inquire whether or not the command processor exists.

If string is a null pointer and the command processor exists, then zero is returned. All other return values are implementation-defined.

2.13.5 Searching and Sorting Functions

2.13.5.1 bsearch

Declaration:

```
void *bsearch(const void *key, const void *base, size_t nitems, size_t size,
int (*compar)(const void *, const void *));
```

Performs a binary search. The beginning of the array is pointed to by *base*. It searches for an element equal to that pointed to by *key*. The array is *nitems* long with each element in the array *size* bytes long.

The method of comparing is specified by the *compar* function. This function takes two arguments, the first is the key pointer and the second is the current element in the array being compared. This function must return less than zero if the compared value is less than the specified key. It must return zero if the compared value is equal to the specified key. It must return greater than zero if the compared value is greater than the specified key.

The array must be arranged so that elements that compare less than key are first, elements that equal key are next, and elements that are greater than key are last.

If a match is found, a pointer to this match is returned. Otherwise a null pointer is returned. If multiple matching keys are found, which key is returned is unspecified.

2.13.5.2 qsort

Declaration:

```
void qsort(void *base, size_t nitems, size_t size, int (*compar)(const void
*, const void*));
```

Sorts an array. The beginning of the array is pointed to by *base*. The array is *nitems* long with each element in the array *size* bytes long.

The elements are sorted in ascending order according to the *compar* function. This function takes two arguments. These arguments are two elements being compared. This function must return less than zero if the first argument is less than the second. It must return zero if the first argument is equal to the second. It must return greater than zero if the first argument is greater than the second.

If multiple elements are equal, the order they are sorted in the array is unspecified.

No value is returned.

```
Example:

#include<stdlib.h>
#include<stdio.h>
#include<string.h>

int main(void)
{
  char string_array[10][50]={"John",
                             "Jane",
                             "Mary",
                             "Rogery",
                             "Dave",
                             "Paul",
                             "Beavis",
                             "Astro",
                             "George",
                             "Elroy"};

  /* Sort the list */
  qsort(string_array,10,50,strcmp);

  /* Search for the item "Elroy" and print it */
  printf("%s",bsearch("Elroy",string_array,10,50,strcmp));

  return 0;
}
```

2.13.6 Math Functions

2.13.6.1 abs

Declaration:

```
int abs(int x);
```

Returns the absolute value of *x*. Note that in two's compliment that the most maximum number cannot be represented as a positive number. The result in this case is undefined.

The absolute value is returned.

2.13.6.2 div

Declaration:

```
div_t div(int numer, int denom);
```

Divides *numer* (numerator) by *denom* (denominator). The result is stored in the structure `div_t` which has two members:

```
int qout;
int rem;
```

Where *quot* is the quotient and *rem* is the remainder. In the case of inexact division, *quot* is rounded down to the nearest integer. The value *numer* is equal to `quot * denom + rem`.

The value of the division is returned in the structure.

2.13.6.3 labs

Declaration:

```
long int labs(long int x);
```

Returns the absolute value of *x*. Not that in two's compliment that the most maximum number cannot be represented as a positive number. The result in this case is undefined.

The absolute value is returned.

2.13.6.4 ldiv

Declaration:

```
ldiv_t ldiv(long int numer, long int denom);
```

Divides *numer* (numerator) by *denom* (denominator). The result is stored in the structure `ldiv_t` which has two members:

```
long int qout;
long int rem;
```

Where *quot* is the quotient and *rem* is the remainder. In the case of inexact division, *quot* is rounded down to the nearest integer. The value *numer* is equal to `quot * denom + rem`.

The value of the division is returned in the structure.

2.13.6.5 rand

Declaration:

```
int rand(void);
```

Returns a pseudo-random number in the range of `0` to `RAND_MAX`.

The random number is returned.

2.13.6.6 srand

Declaration:

```
void srand(unsigned int seed);
```

This function seeds the random number generator used by the function **rand**. Seeding **srand** with the same seed will cause **rand** to return the same sequence of pseudo-random numbers. If **srand** is not called, **rand** acts as if **srand(1)** has been called.

No value is returned.

2.13.7 Multibyte Functions

The behavior of the multibyte functions are affected by the setting of **LC_CTYPE** in the location settings.

2.13.7.1 mblen

Declaration:

```
int mblen(const char *str, size_t n);
```

Returns the length of a multibyte character pointed to by the argument *str*. At most *n* bytes will be examined.

If *str* is a null pointer, then zero is returned if multibyte characters are not state-dependent (shift state). Otherwise a nonzero value is returned if multibyte character are state-dependent.

If *str* is not null, then the number of bytes that are contained in the multibyte character pointed to by *str* are returned. Zero is returned if str points to a null character. A value of -1 is returned if *str* does not point to a valid multibyte character.

2.13.7.2 mbstowcs

Declaration:

```
size_t mbstowcs(schar_t *pwcs, const char *str, size_t n);
```

Converts the string of multibyte characters pointed to by the argument *str* to the array pointed to by *pwcs*. It stores no more than *n* values into the array. Conversion stops when it reaches the null character or *n* values have been stored. The null character is stored in the array as zero but is not counted in the return value.

If an invalid multibyte character is reached, then the value -1 is returned. Otherwise the number of values stored in the array is returned not including the terminating zero character.

2.13.7.3 mbtowc

Declaration:

```
int mbtowc(whcar_t *pwc, const char *str, size_t n);
```

Examines the multibyte character pointed to by the argument *str*. The value is converted and stored in the argument *pwc* if *pwc* is not null. It scans at most *n* bytes.

If *str* is a null pointer, then zero is returned if multibyte characters are not state-dependent (shift state). Otherwise a nonzero value is returned if multibyte character are state-dependent.

If *str* is not null, then the number of bytes that are contained in the multibyte character pointed to by str are returned. Zero is returned if *str* points to a null character. A value of -1 is returned if *str* does not point to a valid multibyte character.

2.13.7.4 wcstombs

Declaration:

```
size_t wcstombs(char *str, const wchar_t *pwcs, size_t n);
```

Converts the codes stored in the array *pwcs* to multibyte characters and stores them in the string *str*. It copies at most *n* bytes to the string. If a multibyte character overflows the *n* constriction, then none of that multibyte character's bytes are copied. Conversion stops when it reaches the null character or *n* bytes have been written to the string. The null character is stored in the string, but is not counted in the return value.

If an invalid code is reached, the value -1 is returned. Otherwise the number of bytes stored in the string is returned not including the terminating null character.

2.13.7.5 wctomb

Declaration:

```
int wctomb(char *str, wchar_t wchar);
```

Examines the code which corresponds to a multibyte character given by the argument *wchar*. The code is converted to a multibyte character and stored into the string pointed to by the argument *str* if *str* is not null.

If *str* is a null pointer, then zero is returned if multibyte characters are not state-dependent (shift state). Otherwise a nonzero value is returned if multibyte character are state-dependent.

If *str* is not null, then the number of bytes that are contained in the multibyte character wchar are returned. A value of -1 is returned if wchar is not a valid multibyte character.
